

within a vector is referred to as a 'lane'. So, the number of lanes that can be accommodated in a vector is determined by the vector species.

For example, the Float Vector of Species 'VectorSpecies<Float> SPECIES_512' can fit 16 floats (16 lanes of float type; 512-bit width/32-bit float size = 16 floats). Figure 4 shows the number of lanes supported for a platform, which support a 512-bit wide vector register for each data type.

For every data type we can get the preferred vector supported by the platform using the 'SPECIES_PREFERRED' defined in the respective data type vector. For example, for float type we can get the float species preferred by the platform using the following:

```
VectorSpecies<Float> preferredSpecies =
FloatVector.SPECIES_PREFERRED;
```

Let us now understand how to write simple vector code by taking an example provided in the JEP proposal of Vector API.

Consider two float arrays and an operation involving the addition of the squares of two numbers, followed by their negation. In Figure 5, a basic scalar operation — a non-vectorised, sequential code has been implemented (simply $((a^2 + b^2) * (-1))$).

In Figure 6, the vector code has been implemented. The vector species provides an API called 'loopBound', which returns the maximum iterations that can be performed in a vectorized manner for the total provided iterations. Considering that FloatVector.Species_512 can support sixteen floats at a time, for 10000 iterations the loopBound API for FloatVector.Species_512 returns 625 (10000/16). Here, we can fit all the data in vectorized processing for 10000 float elements into 625 vectorized iterations (line number 52 to 61).

Let's take another example where we must process only 36 float values instead of 10000. The loopBound API will return 2 ($36/16 = 2.xx$),

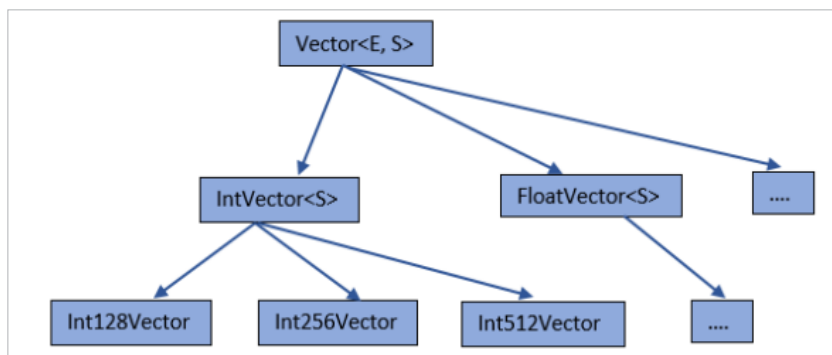


Figure 3: Vector API class hierarchy

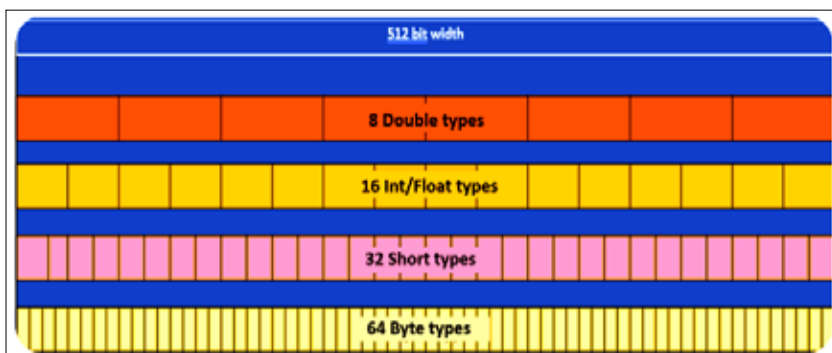


Figure 4: Number of lanes for a 512 bit-width vector

```

public class ScalarAndVectorMulComparision {

    static final VectorSpecies<Float> SPECIES = FloatVector.SPECIES_PREFERRED;

    @State(Scope.Thread)
    public static class StateObj {
        float a[] = new float[1000];
        float b[] = new float[1000];
        float c[] = new float[1000];

        @Setup(Level.Invocation)
        public void setupInvocation() {
            for (int index = 1; index < 1001; index++) {
                a[index] = index;
                b[index] = index;
                c[index] = 0;
            }
        }
    }

    @Benchmark
    @Fork(value = 1)
    @Threads(1)
    @Warmup(iterations = 1, time = 10, timeUnit = TimeUnit.SECONDS)
    @Measurement(iterations = 10, time = 10, timeUnit = TimeUnit.SECONDS)
    @BenchmarkMode(Mode.AverageTime)
    @OutputTimeUnit(TimeUnit.NANOSECONDS)
    public void scalarComputation(ScalarAndVectorMulComparision.StateObj stateObj) {
        for (int i = 0; i < stateObj.a.length; i++) {
            stateObj.c[i] = (stateObj.a[i] * stateObj.a[i] + stateObj.b[i] * stateObj.b[i]) * -1.0f;
        }
    }
}
  
```

Figure 5: Scalar computation